run_cube_root_2_f5

November 6, 2026

[2]:
```
%%time
D = 120
prec = 4*(D + 20); CC = ComplexBallField(prec); RBF = RealBallField(prec); Pi =␣
 ↪CC(pi, 0)
load('cubic_field_K_cube_root_2_f5.sage')
load('BCG_cubic.sage')
prec = 4*(D + 20); CC = ComplexBallField(prec); RBF = RealBallField(prec); Pi =␣
 ↪CC(pi, 0)
L = ff*bb^(-2)
adm = scan_admissible(10, L, 20000, D)
u = EllGammaunit_aa_h(adm/q_ff, L, D)
w = u + 1/u
Q = algdep_K(w, 6, certified_digits(w) - 5)
x = Q.parent().gen()
PK = (x^6 * Q(x + 1/x)).numerator()
print(PK)
print('PK irreducible over K:', PK.is_irreducible())
```

```
Loaded 'cubic_field_K_cube_root_2_f5.sage' (disc -108, f = (5)).

K: Number Field in y with defining polynomial y^3 - 2
K has discriminant  -108 = -1 * 2^2 * 3^3
K has class number  1

pol_K =  y^3 - 2
K_basis_SAGE =  [1, y, y^2]

yRR =  1.25992104989487 …
yCC =  -0.629960524947437 … + I *  1.09112363597172 …
sigmaC = complex embedding of K sending y to yCC

ff =  Fractional ideal (5)  of norm  125
q_ff = 5
aa =  Fractional ideal (y^2 + y - 1)  of norm  11
N_aa =  11
bb =  Fractional ideal (y)  of norm  2  (class generates Cl^+_K(f) = Z/12)

vareps = (beta-1)^8 =  -80*y^2 + 100*y + 1
```

```
varepsRR =  0.00002083203135849 66
Eps =
[   1  100  -80]
[-160    1  100]
[ 200 -160    1]

--- consistency checks ---
disc = -108           : True
class number = 1      : True
N(aa) = 11 prime      : True
sigma_R(vareps) in (0,1): True
vareps - 1 in ff      : True
det(Eps) = +-1        : True
--------------------------

N(aa) = 11 is an odd prime, as (2.9) asks.
aa is coprime to ff, as (2.9) asks.
sigmaR(vareps) = 0.000020832 lies in (0,1), as section 2.2 asks.
vareps lies in 1 + ff, as section 2.2 asks.
q_ff = 5 is a positive integer of ff, as section 2.2 asks.

the five best admissible vectors, with the number of terms they need for 120
decimals:
s/rho, s, rho, N_Ell_Gamma, adm
3.8818e7 5060 0.00013035 2119748 (130, -175/2, -25/2)
2.3291e8 660 2.8337e-6 97508382 (255, -50, -125)
8.9282e8 5060 5.6674e-6 48754191 (80, 125/2, -50)
1.2034e9 5060 4.2049e-6 65712170 (105, -25/2, -100)
1.9797e9 5060 2.5559e-6 108107119 (-245, -125/2, -175/2)
Did not have to change signs of a and b to satisfy Lemma 2.1 (ii).
Evaluating at h = adm/5 = (26, -35/2, -5/2)  with adm = (130, -175/2, -25/2)
The wedge for  (26, -35/2, -5/2)  is
a =  (9, 94/5, -38)
b =  (-271, -1706/5, -430)
alpha = (16125, -10750, -3265/2) , alpha(b) =  0 alpha(a) =  5060
beta = (-1425, 950, 265/2) , beta(a) =  0 beta(b) =  5060
gamma = (-130, 175/2, 25/2)
(alpha,beta,gamma) is positively oriented basis
tau =  -123.63863632 - 0.99586697526*I
sigma =  10.913636222 + 0.000020745932058*I
The set F has  5060 elements.
Wedge negatively oriented with respect to h
certified decimals =  120
u =  0.00076584400811 - 0.0013776204621*I
PK is irreducible over K.
PK irreducible over K: True
CPU times: user 2min 13s, sys: 28.9 ms, total: 2min 13s
Wall time: 2min 13s
```

[3]:
```
print(PK)
```

```
x^12 + (22480*y^2 + 29990*y - 73467)*x^11 + (66020030*y^2 - 498291665*y +
523007896)*x^10 + (164504568365*y^2 + 79694900795*y - 361544007955)*x^9 +
(18592531801350*y^2 - 28337852637215*y + 6189652506230)*x^8 +
(-657941621539610*y^2 - 638642322605235*y + 1849056127769723)*x^7 +
(409191912449815*y^2 + 1982206764789930*y - 3146975700482571)*x^6 +
(-657941621539610*y^2 - 638642322605235*y + 1849056127769723)*x^5 +
(18592531801350*y^2 - 28337852637215*y + 6189652506230)*x^4 + (164504568365*y^2
+ 79694900795*y - 361544007955)*x^3 + (66020030*y^2 - 498291665*y +
523007896)*x^2 + (22480*y^2 + 29990*y - 73467)*x + 1
```

[22]:
```
d = K.discriminant()
print('Test discriminant is square ?')
print('   disc(K) =', d, '=', factor(d), ' ; non square part is', d.
 ↪squarefree_part())
print('   disc(K) is a square in Q(sqrt(-3)) :', QuadraticField(-3)(d).
 ↪is_square())
```

```
Test discriminant is square ?
   disc(K) = -108 = -1 * 2^2 * 3^3  ; non square part is -3
   disc(K) is a square in Q(sqrt(-3)) : True
```

[23]:
```
bnf = K.pari_bnf()
print('Test u generates class field ?')
print('   PK irreductible over K :', PK.is_irreducible())
print('   K(u)/K abelian :', bnf.rnfisabelian(PK) == 1)
print('   conductor of K(u)/K, ideal  and real places :', bnf.
 ↪rnfconductor(PK)[0])
print('   narrow class number mod 5 :', bnf.bnrclassno([5, [1]]), ' = degree of␣
 ↪PK :', PK.degree())
Ku.<u> = K.extension(PK)
P36 = u.absolute_minpoly()
print('   minimal polynomial of  u over Q : degree', P36.degree(), ' = [K(u) :␣
 ↪Q] =', Ku.absolute_degree(), ', so that  Q(u) = K(u) contains K')
```

```
Test u generates class field ?
   PK irreductible over K : True
   K(u)/K abelian : True
   conductor of K(u)/K, ideal  and real places : [[5, 0, 0; 0, 5, 0; 0, 0, 5],
[1]]
   narrow class number mod 5 : 12  = degree of PK : 12
   minimal polynomial of  u over Q : degree 36  = [K(u) : Q] = 36 , so that
Q(u) = K(u) contains K
```

[25]:
```
T = pari(P36).polredbest()
C = T.polcompositum(pari('x^2 + 3'))[0]
L = NumberField(QQ['x'](C), 'z', maximize_at_primes=[5])
```

```
ef = []
for P in L.primes_above(5):
   ef.append((P.ramification_index(), P.residue_class_degree()))
print('Test is L/k0 Galois ?')
print('  degree of L', L.degree(), ' ; 5 is inert in  k0 :', QuadraticField(-3).
 ↪ideal(5).is_prime())
print('   (e, f) of primes of  L above 5 :', ef)
print('   all the same, as should be if L/k0 is to be Galois:', len(set(ef)) ==␣
 ↪1)
```

```
Test is L/k0 Galois ?
  degree of L 72  ; 5 is inert in  k0 : True
   (e, f) of primes of  L above 5 : [(12, 2), (12, 2), (4, 6)]
   all the same, as should be if L/k0 is to be Galois: False
```

[ ]: